CSE470: Software Engineering

Software Requirements Specification (SRS)

Software Requirements Specification (SRS) — Draft for Review

Campus Project & Thesis Management System (CPTMS)

Prepared By

Student ID	Name
Sabab Al Saba	24101529
Walid Mahmood	22201456
Md Iftekhar Islam Abir	21301109

Project Information

Section	13
Section Faculty	Mr. Soumya Swagata Biswas [SOSB]
GitHub Repository Link	TBA
Tech Stack Used	TBA

1. Introduction

1.1 Purpose

This Software Requirements Specification defines the proposed requirements, users, architecture, interfaces, and verification approach for the Campus Project & Thesis Management System (CPTMS). It provides a shared baseline for implementation and faculty review; requirements and feature scope remain subject to faculty approval.

1.2 Scope

CPTMS is a browser-based system for university students, faculty supervisors, project evaluators, and department coordinators to manage academic course projects and theses from team formation and proposal submission through supervision, milestones, evaluation, and archiving. The initial scope includes project records, workflow approvals, scheduling, progress tracking, evaluations, notifications, and department-level reporting. Out of scope are AI/ML features, a native mobile application, automatic plagiarism detection, payment processing, and replacement of official university student-information systems.

2. Overall Description

2.1 Product Perspective

CPTMS is a proposed web application that centralizes a workflow often coordinated through separate email threads, spreadsheets, documents, and meetings. It is intended to support an academic department rather than replace the university's official student-information system. Integration with external university systems is not assumed for the initial version.

2.2 Product High-Level Summary

The system shall support team formation and eligibility checks; topic discovery; supervisor capacity and assignment; structured proposal submission, review, revision, and approval; milestone and deliverable tracking; meeting scheduling and notifications; team-change requests; evaluation rubrics and defense scheduling; grade calculation; peer contribution assessment; project archiving; departmental analytics; and an audit trail. Each requirement is expanded into a distinct feature in Section 4, with one feature representing a coherent workflow rather than separate CRUD actions.

2.3 Users and Characteristics

- Student / Team Member: creates or joins a team, explores topics, submits proposals and deliverables, views feedback and deadlines, requests changes, and completes peer assessments.
- Faculty Supervisor: declares interests and capacity, reviews proposals and deliverables, gives feedback, tracks progress, and records supervision meetings.
- Project Evaluator: reviews assigned projects, participates in defenses, and scores work using approved rubrics.
- Department Coordinator / Administrator: configures academic rules and deadlines, resolves assignments and escalations, schedules defenses, manages evaluation workflows, and views department reports.
- System Administrator (if separated from the coordinator role): manages accounts, roles, and system configuration. Access is restricted by role and authorization rules.

2.4 Operating Environment

The application is intended to run on current desktop and mobile web browsers. Development requires a supported .NET SDK, Visual Studio or Visual Studio Code, SQL Server or SQL Server Express, and Entity Framework Core tooling. The server environment must support ASP.NET Core; users require network access and a modern browser. Exact deployment hosting is to be selected by the team and approved if required.

2.5 Design and Implementation Constraints

This project follows the constraints defined in the CSE470 Project Outline, including mandatory MVC architecture and the approved tech-stack policy. State your specific choices below.

Primary stack: C#, ASP.NET Core MVC, Razor Views, Entity Framework Core, SQL Server, HTML/CSS, Bootstrap, and GitHub. MVC organization: Models (domain entities and validation), Views (Razor pages/templates and presentation), and Controllers (request handling, authorization, validation, and workflow coordination). No AI/ML-backed feature or separate Python/FastAPI service is planned. No faculty-approved exception is currently assumed. Authentication may use ASP.NET Core Identity, subject to the team's final design.

2.6 Assumptions and Dependencies (Optional)

Assumptions: the department will provide or approve team-size rules, eligibility criteria, academic deadlines, proposal states, rubric weights, and grading rules. Faculty and students will have individual accounts and role assignments. Users will upload only permitted academic documents. Email or SMS delivery is not required for the first version; in-application notifications are the baseline. Dependencies include the .NET SDK, ASP.NET Core MVC, Entity Framework Core provider for SQL Server, SQL Server, Bootstrap, and a GitHub repository.

3. System Architecture

3.1 Architectural Overview

Request flow: Browser / Razor View → MVC Controller → validation and workflow rules → Entity Framework Core / Model layer → SQL Server. The Controller returns a View or redirects after a successful operation; the View renders the resulting data. Controllers should remain focused on request coordination, while domain rules and data access are kept in models/services/repositories as appropriate. Authorization is checked before restricted actions.

3.2 Technology Stack

- Presentation: Razor Views, HTML5, CSS, Bootstrap, JavaScript where needed.
- Application/backend: C# with ASP.NET Core MVC.
- Domain and persistence: C# model classes and Entity Framework Core.
- Database: Microsoft SQL Server / SQL Server Express.
- Authentication/authorization: ASP.NET Core Identity and role-based authorization (planned; confirm during implementation).
- Version control and collaboration: Git and GitHub.
- Testing: xUnit and ASP.NET Core integration testing where practical.
- Hosting/deployment: to be selected; local development on Windows is the initial target.

3.3 Class Diagram

Editable conceptual class diagram (revise to match the final implemented model):

```
User (UserId, Name, Email, Role)
  ├── StudentProfile (UserId, StudentId, Department, CGPA, Semester)
  ├── FacultyProfile (UserId, Department, Capacity, Interests)
  └── EvaluationAssignment (EvaluatorId, ProjectId, DefenseSlotId)

ProjectTeam (TeamId, Name, Department, CreatedByStudentId)
  ├── TeamMembership (TeamId, StudentId, MembershipRole, Status)
  └── Project (ProjectId, TeamId, Title, Type, Status, SupervisorId)
      ├── Proposal (ProposalId, ProjectId, VersionNo, SubmittedAt, DecisionStatus)
```

```
│   └── ProposalReview (ReviewId, ProposalId, ReviewerId, Decision, Feedback)
├── Milestone (MilestoneId, ProjectId, Title, DueDate, Status)
│   └── Deliverable (DeliverableId, MilestoneId, SubmittedBy, FilePath, ReviewStatus)
├── Meeting (MeetingId, ProjectId, SupervisorId, StartAt, EndAt, Status)
├── Evaluation (EvaluationId, ProjectId, EvaluatorId, RubricId, TotalScore)
│   └── EvaluationScore (EvaluationId, CriterionId, Score, Comment)
├── PeerAssessment (AssessmentId, ProjectId, ReviewerStudentId, TargetStudentId, Score, Comment)
└── ProjectArchive (ArchiveId, ProjectId, ArchivedAt, FinalReportPath)
```

Supporting entities: Topic (TopicId, Title, Domain, Capacity, FacultyOwnerId), Rubric (RubricId, Name, Version), RubricCriterion (CriterionId, RubricId, Weight, MaxScore), DefenseSlot (DefenseSlotId, Room, StartAt, EndAt), Notification (NotificationId, UserId, Message, ReadAt), ChangeRequest (RequestId, ProjectId, RequestType, Status), and AuditLog (LogId, ActorId, Action, EntityType, EntityId, Timestamp). A Project is associated with one team and may have one assigned supervisor at a time; a project has multiple proposals/versions, milestones, deliverables, meetings, and evaluations. Final cardinalities and normalization must be confirmed during database design.

4. Requirements and Features

The following are proposed substantive features, not claims of implementation. Each row describes one coherent capability; related create/read/update actions are grouped together. Confirm the scope and feature count with the section faculty before Sprint 1.

Status is initially Planned because implementation and sprint reviews have not yet been reported. Update the sprint and status columns only when supported by actual work and evidence.

Req. ID	Requirement	Feature Description	Actor	MVC Components Touched	Sprint Completed	Status
R1	Students can form and manage project teams subject to membership rules.	F1 — Create a team, invite/request membership, accept or reject requests, and manage member status within configured team-size limits.	Student / Team Member	Model: ProjectTeam, TeamMembership; View: Teams; Controller: TeamsController	Sprint 1	Planned
R2	Students can discover available project topics.	F2 — Search, filter, and view available topics by domain, skills, project type, and availability.	Student	Model: Topic; View: Topics/Index, Topics/Details; Controller: TopicsController	Sprint 1	Planned
R3	Faculty can declare supervision capacity and interests.	F3 — Maintain faculty interests, eligibility conditions, and maximum supervision load; reject capacity-increasing changes that exceed approved limits.	Faculty Supervisor / Coordinator	Model: FacultyProfile, Topic; View: Faculty/Availability; Controller: FacultyController	Sprint 2	Planned
R4	The department can assign eligible supervisors to project teams.	F4 — Record team preferences and assign a supervisor using eligibility, declared capacity, and coordinator decision; prevent assignments that exceed capacity.	Coordinator	Model: Project, FacultyProfile; View: Assignments; Controller: AssignmentsController	Sprint 2	Planned
R5	The system validates team and project eligibility.	F5 — Apply configurable rules such as team-size range, department, semester, CGPA threshold, and project type before allowing	Student / Coordinator	Model: TeamMembership, StudentProfile, EligibilityRule; View: Eligibility; Controller: EligibilityController	Sprint 1	Planned

Req. ID	Requirement	Feature Description	Actor	MVC Components Touched	Sprint Completed	Status
		submission or assignment.				
R6	The system flags potentially duplicate project topics.	F6 — Compare normalized titles and selected keywords/categories against existing projects and flag possible overlap for human review; do not automatically reject a proposal.	Coordinator / Supervisor	Model: Project, Topic; View: SimilarProjects; Controller: ProjectsController	Sprint 2	Planned
R7	Teams can submit structured project proposals.	F7 — Submit a proposal containing title, problem statement, objectives, methodology, expected outcomes, technology plan, project type, and team details.	Student / Team Member	Model: Proposal, Project; View: Proposals/Create; Controller: ProposalsController	Sprint 2	Planned
R8	Proposals follow an authorized multi-stage approval workflow.	F8 — Route proposals through supervisor review and department approval, enforce permitted state transitions, and record each decision.	Supervisor / Coordinator	Model: Proposal, ProposalReview; View: Proposals/Review; Controller: ProposalsController	Sprint 2	Planned
R9	Reviewers can request revisions and teams can resubmit.	F9 — Attach actionable feedback to a proposal, return it for revision, and allow resubmission while preserving review history.	Student / Supervisor / Coordinator	Model: Proposal, ProposalReview; View: Proposals/Details; Controller: ProposalsController	Sprint 2	Planned
R10	The system preserves proposal version and decision history.	F10 — Keep prior submitted versions, timestamps, reviewers, decisions, and feedback accessible to authorized users.	Student / Supervisor / Coordinator	Model: Proposal, ProposalReview; View: Proposals/History; Controller: ProposalsController	Sprint 2	Planned
R11	The system enforces proposal submission windows and prerequisites.	F11 — Validate deadlines, late-submission policy, required fields, and prerequisite approvals before accepting a proposal.	Student / Coordinator	Model: Proposal, DeadlinePolicy; View: Proposals/Create; Controller: ProposalsController	Sprint 2	Planned
R12	Approved projects can define milestones and deliverables.	F12 — Create and maintain a project plan containing milestones, due dates, expected deliverables, and dependencies.	Supervisor / Team	Model: Project, Milestone; View: Milestones; Controller: MilestonesController	Sprint 3	Planned
R13	Teams can submit deliverables for review.	F13 — Submit files or links against a milestone, validate permitted file types/size, and allow reviewers to accept or request resubmission with feedback.	Student / Supervisor	Model: Deliverable, Milestone; View: Deliverables; Controller: DeliverablesController	Sprint 3	Planned
R14	Users can track project progress and blockers.	F14 — Display milestone status and calculate pending, overdue, blocked, and completed work from recorded dates and states.	Student / Supervisor / Coordinator	Model: Project, Milestone, Deliverable; View: Projects/Progress; Controller: ProgressController	Sprint 3	Planned
R15	Teams and supervisors can schedule consultations.	F15 — Request, accept, reschedule, or cancel consultation meetings and prevent overlapping bookings for the same participant.	Student / Supervisor	Model: Meeting, FacultyProfile; View: Meetings; Controller: MeetingsController	Sprint 3	Planned

Req. ID	Requirement	Feature Description	Actor	MVC Components Touched	Sprint Completed	Status
R16	The system provides deadline and workflow notifications.	F16 — Notify relevant users about upcoming deadlines, review decisions, requested revisions, overdue milestones, and meeting changes through in-app notifications.	Student / Supervisor / Coordinator	Model: Notification; View: Notifications; Controller: NotificationsController	Sprint 3	Planned
R17	Teams can request membership or supervisor changes and report disputes.	F17 — Submit a reasoned change request, route it to authorized reviewers, record the decision, and preserve request history.	Student / Coordinator / Supervisor	Model: TeamMembership, ChangeRequest; View: ChangeRequests; Controller: ChangeRequestsController	Sprint 3	Planned
R18	Faculty can define evaluation rubrics.	F18 — Configure evaluation criteria, weights, maximum scores, and rubric versions; validate that weights follow the department's approved rule.	Coordinator / Evaluator	Model: Rubric, RubricCriterion; View: Rubrics; Controller: RubricsController	Sprint 3	Planned
R19	The department can schedule project presentations and defenses.	F19 — Assign defense date/time, room, project, and evaluators while checking room and participant conflicts.	Coordinator	Model: DefenseSlot, EvaluationAssignment; View: Defenses; Controller: DefensesController	Sprint 3	Planned
R20	Evaluators can score projects and calculate final evaluation results.	F20 — Record criterion-level scores and comments, validate score ranges, and calculate weighted totals using the selected rubric.	Evaluator / Coordinator	Model: Evaluation, EvaluationScore, RubricCriterion; View: Evaluations; Controller: EvaluationsController	Sprint 3	Planned
R21	Students can provide structured peer contribution assessments.	F21 — Submit confidential assessments for eligible teammates using approved criteria, restrict access to authorized roles, and summarize results according to department policy.	Student / Coordinator	Model: PeerAssessment; View: PeerAssessments; Controller: PeerAssessmentsController	Sprint 4	Planned
R22	Completed projects can be finalized and archived.	F22 — Verify required approvals and deliverables, record completion, and make the final report and project metadata searchable to authorized users.	Coordinator / Supervisor	Model: ProjectArchive, Project, Deliverable; View: Archive; Controller: ArchiveController	Sprint 4	Planned
R23	Department staff can view project workflow analytics.	F23 — Produce filtered summaries of project status, completion rates, supervisor workload, approval delays, evaluation outcomes, and overdue milestones from stored records.	Coordinator	Model: Project, Milestone, Proposal, Evaluation; View: Reports; Controller: ReportsController	Sprint 4	Planned
R24	The system records important actions and decisions.	F24 — Maintain an audit trail for assignments, approvals, status changes, evaluation updates, and administrative actions with actor and timestamp.	Coordinator / Administrator	Model: AuditLog; View: AuditLogs; Controller: relevant workflow controllers / AuditLog service	Sprint 4	Planned

4.1 Feature Count Summary

Minimum features required (per Project Outline, Section 7)	20
Total features listed in Section 4 above	24
Total features fully completed (“Done” per Section 7.2 of the Project Outline)	0 (not yet implemented / verified)

5. External Interface Requirements (Optional)

Interface Type	Description
User Interface	Responsive browser-based interface using Razor Views and Bootstrap. Role-specific dashboards, forms, lists, review pages, and status/history views.
Hardware Interface	No dedicated hardware interface is required. A computer or mobile device with a supported browser and network connection is sufficient.
Software Interface (APIs, third-party services)	ASP.NET Core MVC application with Entity Framework Core and SQL Server. No external API is required for the initial scope. Email/SMS integration is optional and excluded unless approved and implemented.
Communication Interface	HTTPS for deployed environments; local development may use localhost. Browser requests and server responses use standard HTTP/HTTPS. File uploads must be validated and access-controlled.

6. Non-Functional Requirements (Optional)

Category	Requirement
Performance	Common pages should respond within an agreed target (proposed: under 3 seconds for typical requests with a small course-sized dataset), excluding large uploads and network delays. Verify through repeatable tests before claiming compliance.
Security	Use authenticated accounts, role-based authorization, server-side validation, safe password handling through an established identity framework, anti-forgery protection, and restricted file access. Do not rely on hiding UI controls as authorization.
Usability	Use consistent navigation, clear validation messages, readable status labels, and forms that explain required fields. Core workflows should be usable in current desktop and mobile browsers.
Reliability / Availability	Persist changes transactionally where appropriate, show actionable errors without exposing secrets, and prevent invalid workflow transitions. Backups and production availability targets depend on the selected host.
Scalability	Use indexed relational data and paginated/filterable lists where appropriate. Initial scale is a department/course cohort; larger deployment requires performance review and load testing.

7. Sprint History and Review Ceremony Log

The goals below are a proposed four-week plan. Record the actual goals at the start of each week and actual attainment at the end of each sprint; do not report planned work as completed.

Sprint	Goals Set (Start of Week)	Attainment Reported (End of Week)	Features Completed This Sprint
Sprint 1	Finalize faculty-approved SRS and acceptance criteria; design ER model; create ASP.NET Core MVC/EF Core solution; establish GitHub workflow and role model; implement team formation and eligibility baseline.	Complete after actual sprint review.	F1, F5 (planned)
Sprint 2		.	
Sprint 3			
Sprint 4			

7.1 Deviations and Lessons Learned

Complete this section after sprint reviews. For each deviation, state the planned goal, actual outcome, reason, and the adjustment made for the next sprint.

8. Testing Summary

Planned verification approach (replace with actual results after implementation): test role authorization for every restricted workflow; test valid and invalid team membership and eligibility; test proposal state transitions and resubmission; test supervisor capacity limits and scheduling conflicts; test deadline and overdue status calculations; test score range and weighted grade calculations; test upload validation; and run integration tests for the proposal-to-archive workflow. Record test cases, expected results, actual results, and defects fixed. No test is marked passed until it has actually been executed.

9. AI Tool Usage Disclosure

Generative AI was used to help draft and organize this SRS and propose initial requirement descriptions.

Feature / Task	AI Tool Used	How It Was Used (be specific)	I Can Fully Explain This Code (Yes/No)
SRS draft and initial feature descriptions	ChatGPT	Assisted with organizing this editable SRS draft and proposing initial requirement/feature wording.	N/A — documentation only

[Before submission, each student must confirm they can personally explain, modify, and debug the code they contributed and can explain the overall architecture and feature workflow.]

10. Individual Contribution Summary

Complete this section individually after implementation. For each student, list the feature IDs personally implemented, approximate effort per sprint, integration/testing/documentation work, and challenges solved. If the course requires individual implementation despite group planning, clearly separate individual coding from group discussion or coordination.

11. Pre-Submission Checklist

Checklist Item	Confirmed (Yes/No)
The application implements the MVC architecture	
At least 20 features are listed and each is correctly counted per the granularity rule	
No forbidden framework (e.g. Django, Flask as primary backend) was used	
All four sprints' goals and attainment are recorded in Section 7	
The GitHub repository is up to date and the section faculty is added as a collaborator	
The AI Usage Disclosure table (Section 10) is complete and honest	
I can personally explain and defend every feature listed in this document during the viva	